

Secure Command Communication System (RSA + HMAC)

Dhruv Shukla

Dipanshu Teotia

ABSTRACT

This project presents a secure communication protocol for transmitting control commands over an insecure network using RSA encryption and HMAC-based authentication. The system ensures confidentiality, integrity, and protection against replay attacks.

A secure session key is established using RSA encryption, after which control commands are transmitted along with a counter and HMAC for authentication. The HMAC ensures that the message has not been altered, while the counter prevents replay attacks by ensuring message freshness.

To enhance security, adaptive key evolution is implemented, where the session key is updated after each valid message. This prevents reuse of compromised keys and strengthens the system against attacks.

An attacker model is also included to simulate real-world threats such as packet interception, tampering, and replay attacks. The system successfully detects and rejects invalid or malicious messages, ensuring that only authentic commands are executed.

This project demonstrates a practical and secure approach to command communication, suitable for applications such as IoT systems, remote systems, and industrial control environments.

TABLE OF CONTENTS

INTRODUCTION	4
PROBLEM STATEMENT	5
OBJECTIVES	5
LITERATURE REVIEW	6
SYSTEM ARCHITECTURE	7
METHODOLOGY	8
ALGORITHMS USED	10
RSA (Rivest–Shamir–Adleman)	10
HMAC (Hash-based Message Authentication Code)	10
Counter-Based Replay Protection	10
Key Evolution Mechanism	10
IMPLEMENTATION DETAILS	12
Technologies Used	12
Module Description	12
Cryptographic Module (crypto_utils.py)	12
Client Module (secure_client.py)	12
Server Module (secure_server.py)	12
Attacker Module (attacker.py)	13
Working Overview	13
RESULTS AND OUTPUTS	14
1. Normal Communication	14
2. Replay Attack Detection	14
3. Key Evolution	15
SECURITY ANALYSIS	16
1. Confidentiality	16
2. Message Integrity	16
3. Authentication	16
4. Replay Attack Prevention	16
5. Key Evolution and Forward Security	16
6. Resistance to Man-in-the-Middle Attacks	16
ADVANTAGES AND LIMITATIONS	17
Advantages	17
Limitations	17
FUTURE SCOPE	18
CONCLUSION	19
REFERENCES	20

INTRODUCTION

With the rapid growth of network-based systems, secure communication has become essential. Control commands such as starting, stopping, or managing devices are highly sensitive and require strong protection against unauthorized access.

In insecure environments, attackers can intercept, modify, or replay commands. These attacks can lead to serious consequences, including unauthorized control of systems and operational failures. Therefore, ensuring the security of control command communication is critical.

This project focuses on designing a secure communication protocol using RSA encryption and HMAC authentication. RSA is used to securely exchange a session key, while HMAC ensures message integrity and authenticity.

Additionally, a counter mechanism is implemented to prevent replay attacks by ensuring that each message is fresh and unique. The system also uses adaptive key evolution, where the session key is updated after each valid communication, enhancing overall security.

An attacker model is incorporated to simulate real-world threats such as man-in-the-middle attacks, packet tampering, and replay attacks. The system is capable of detecting and rejecting malicious messages, ensuring secure and reliable communication.

PROBLEM STATEMENT

In many communication systems, control commands are transmitted without sufficient security mechanisms. This makes them vulnerable to various attacks such as interception, modification, and replay. Attackers can exploit these weaknesses to execute unauthorized commands, disrupt operations, or compromise the system.

Traditional communication methods often lack proper authentication and integrity verification, allowing malicious entities to tamper with messages without detection. Additionally, the absence of freshness mechanisms enables replay attacks, where previously captured valid messages can be resent to trigger unintended actions.

Therefore, there is a need for a secure communication protocol that ensures confidentiality, integrity, authentication, and protection against replay attacks for control command transmission.

OBJECTIVES

The main objectives of this project are:

- To design a secure protocol for transmitting control commands over a network
- To ensure confidentiality using RSA encryption
- To ensure message integrity and authentication using HMAC
- To prevent replay attacks using a counter-based mechanism
- To implement adaptive key evolution for enhanced security
- To simulate attacker behavior such as interception, tampering, and replay attacks
- To validate the system against real-world security threats

LITERATURE REVIEW

Secure communication has been a fundamental area of research in cryptography and network security. Various techniques have been developed to ensure confidentiality, integrity, and authentication of data transmitted over insecure networks.

RSA is one of the most widely used asymmetric encryption algorithms. It enables secure key exchange by using a pair of public and private keys. Due to its strong mathematical foundation, RSA is commonly used in secure communication protocols to protect sensitive information during transmission.

HMAC (Hash-based Message Authentication Code) is a widely used mechanism for ensuring data integrity and authentication. It combines a cryptographic hash function with a secret key to generate a unique authentication code for each message. HMAC is resistant to tampering and ensures that the message has not been altered during transmission.

Replay attacks are a common threat in network communication, where an attacker captures a valid message and retransmits it to produce unauthorized effects. To prevent such attacks, mechanisms such as counters, timestamps, or nonces are used to ensure message freshness.

Modern secure communication systems often use a combination of encryption and authentication techniques to provide layered security. While encryption ensures confidentiality, authentication mechanisms like HMAC ensure trust and integrity of the data.

In addition, key management plays a critical role in maintaining security. Techniques such as key evolution or key rotation are used to periodically update cryptographic keys, reducing the risk of key compromise and enhancing overall system security.

This project builds upon these established concepts by integrating RSA for key exchange, HMAC for authentication, counter-based replay protection, and adaptive key evolution into a unified secure communication protocol.

SYSTEM ARCHITECTURE

The system architecture consists of three main components: the client, the server, and the attacker. These components interact over an insecure network, where security mechanisms are applied to ensure safe communication.

The client is responsible for generating and sending control commands. It first establishes a secure session by generating a random session key and encrypting it using the server's RSA public key. This encrypted key is then transmitted to the server to securely share the session key.

Once the session is established, the client sends commands along with a counter and an HMAC value. The counter ensures message freshness, while the HMAC guarantees the integrity and authenticity of the message.

The server receives the encrypted session key and decrypts it using its private key to establish the secure session. For each incoming message, the server checks the counter to detect replay attacks and verifies the HMAC to ensure that the message has not been altered. Only valid messages are accepted and executed.

An attacker is also included in the system to simulate real-world threats. The attacker can intercept, modify, or replay packets transmitted between the client and server. The system is designed to detect and reject such malicious activities.

Additionally, the system implements adaptive key evolution, where the session key is updated after each successful message exchange. This enhances security by ensuring that old keys cannot be reused.

This architecture demonstrates a layered security approach, combining encryption, authentication, replay protection, and key management to ensure secure command communication.

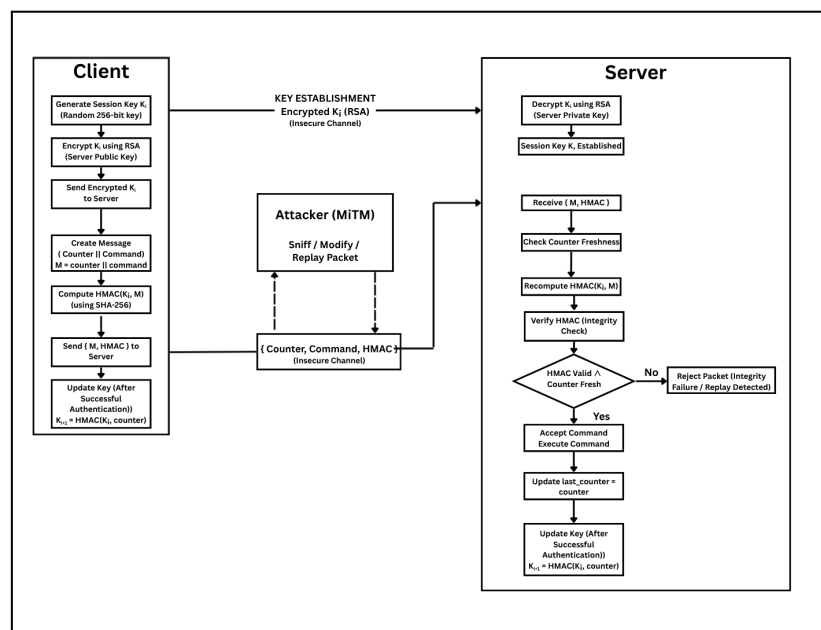


Figure 1: Secure Control Command Communication Architecture

METHODOLOGY

The proposed system follows a structured approach to ensure secure communication of control commands between the client and the server. The methodology consists of the following steps:

1. Session Key Generation

The client generates a random 256-bit session key. This key is used for HMAC-based authentication of messages.

2. Secure Key Exchange using RSA

The generated session key is encrypted using the server's RSA public key and transmitted over the network. The server decrypts this key using its private key to establish a secure session.

3. Message Formation

Each command is combined with a counter value to form the message. The counter ensures that every message is unique and helps in preventing replay attacks.

Message format: $M = \text{counter} \parallel \text{command}$

4. HMAC Generation

The client computes an HMAC using the session key and the message. This ensures message integrity and authentication.

$\text{HMAC} = \text{HMAC}(K_i, M)$

5. Message Transmission

The client sends the message along with the HMAC to the server over an insecure channel.

6. Counter Verification (Replay Protection)

Upon receiving the message, the server checks whether the counter value is greater than the previously received counter. This prevents replay attacks.

7. HMAC Verification

The server recomputes the HMAC using the received message and session key and compares it with the received HMAC. If they match, the message is considered authentic and unmodified.

8. Command Execution

If both the counter verification and HMAC verification are successful, the server accepts and executes the command. Otherwise, the message is rejected.

9. Key Evolution

After successful authentication and execution, the session key is updated using HMAC-based key evolution:

$K_{i+1} = \text{HMAC}(K_i, \text{counter})$

This ensures forward security and prevents reuse of old keys.

This step-by-step methodology ensures confidentiality, integrity, authentication, and protection against replay attacks in the communication system.

ALGORITHMS USED

1. RSA (Rivest–Shamir–Adleman)

RSA is an asymmetric encryption algorithm used for secure key exchange. It uses a pair of keys: a public key for encryption and a private key for decryption.

In this project, RSA is used to securely transmit the session key from the client to the server. The client encrypts the session key using the server's public key, and the server decrypts it using its private key. This ensures that only the intended server can access the session key.

RSA provides strong security based on the mathematical difficulty of factoring large prime numbers. In this implementation, RSA with 2048-bit keys and OAEP padding is used to enhance security.

2. HMAC (Hash-based Message Authentication Code)

HMAC is a cryptographic technique used to ensure data integrity and authentication. It combines a secret key with a hash function to produce a message authentication code.

In this project, HMAC with SHA-256 is used to verify that the message has not been altered during transmission and that it originates from a trusted source. The client generates an HMAC for each message using the session key, and the server verifies it upon receipt.

HMAC is resistant to tampering and ensures that any modification in the message results in a different authentication code, allowing the server to detect malicious changes.

3. Counter-Based Replay Protection

A counter-based mechanism is used to prevent replay attacks. Each message sent by the client includes a counter value that increases with every transmission.

The server maintains the last received counter value and ensures that each new message has a higher counter. If an old or repeated counter is detected, the message is rejected.

This mechanism ensures message freshness and prevents attackers from reusing previously captured valid messages.

4. Key Evolution Mechanism

Key evolution is implemented to enhance security by updating the session key after each successful message exchange.

The new key is generated using the previous key and the counter value:

$$K_{i+1} = \text{HMAC}(K_i, \text{counter})$$

This ensures that even if a key is compromised, it cannot be used to decrypt future messages. Key evolution provides forward security and strengthens the overall robustness of the system.

IMPLEMENTATION DETAILS

The proposed system is implemented using Python and consists of multiple modules that work together to provide secure communication between the client and the server. The implementation focuses on encryption, authentication, replay protection, and attacker simulation.

Technologies Used

- Programming Language: Python
- Cryptographic Library: cryptography
- Networking: socket programming
- Hashing and Authentication: hmac and hashlib
- Packet Analysis: scapy

Module Description

1. Cryptographic Module (crypto_utils.py)

This module is responsible for all cryptographic operations. It implements RSA key generation using 2048-bit keys and performs encryption and decryption using OAEP padding for enhanced security. The module also provides functions for generating and verifying HMAC using SHA-256. Secure comparison methods are used to prevent timing attacks during HMAC verification.

2. Client Module (secure_client.py)

The client initiates communication with the server. It receives the server's public key and generates a random 256-bit session key using a secure random generator. This session key is encrypted using RSA and transmitted securely to the server.

The client sends control commands along with a counter and HMAC. The counter ensures message freshness, while the HMAC ensures integrity and authentication. The client updates the session key using key evolution only after receiving confirmation of successful command execution from the server, ensuring synchronization.

3. Server Module (secure_server.py)

The server generates RSA key pairs and listens for incoming client connections. Upon connection, it securely receives and decrypts the session key. For each incoming message, the server performs counter validation to detect replay attacks and verifies the HMAC to ensure message integrity.

If the message is valid, the server executes the command and sends a response back to the client. Key evolution is performed only after successful authentication, maintaining synchronization between the client and server.

The server follows a concurrent design using threading, allowing it to handle multiple client connections simultaneously.

4. Attacker Module (attacker.py)

The attacker module simulates a man-in-the-middle (MITM) scenario. It uses packet sniffing to capture communication between the client and server. The attacker can replay captured packets to test the system's resistance against replay attacks.

This module is used to validate the robustness of the implemented security mechanisms under adversarial conditions.

Working Overview

The system establishes a secure session using RSA-based key exchange. Once the session key is shared, all subsequent communication is authenticated using HMAC and protected using a counter mechanism to prevent replay attacks.

The implementation ensures that only valid and authenticated commands are executed, while any tampered or replayed messages are detected and rejected. The use of adaptive key evolution further strengthens the security by ensuring that session keys are updated dynamically after each successful communication.

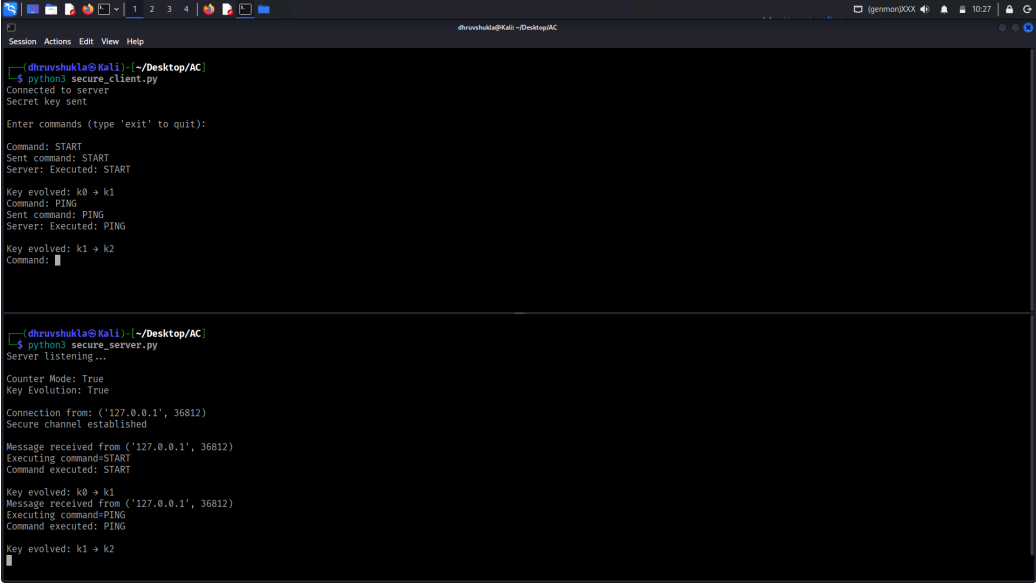
The modular structure of the system allows for easy scalability, testing, and future enhancements.

RESULTS AND OUTPUTS

The implemented system was tested under different scenarios to evaluate its effectiveness in ensuring secure control command communication. The results demonstrate that the system successfully provides confidentiality, integrity, and protection against replay attacks.

1. Normal Communication

In this scenario, the client sends a valid control command along with a correct counter and HMAC. The server verifies the message successfully and executes the command. This confirms that the communication process works correctly under normal conditions.



```
dhruvshukla@kali: ~/Desktop/AC
$ python3 secure_client.py
Connected to server
Secret key sent
Enter commands (type 'exit' to quit):
Command: START
Sent command: START
Server: Executed: START
Key evolved: k0 → k1
Command: PING
Sent command: PING
Server: Executed: PING
Key evolved: k1 → k2
Command:

dhruvshukla@kali: ~/Desktop/AC
$ python3 secure_server.py
Server listening...
Counter Mode: True
Key Evolution: True
Connection from: ('127.0.0.1', 36812)
Secure channel established
Message received from ('127.0.0.1', 36812)
Executing command=START
Command executed: START
Key evolved: k0 → k1
Message received from ('127.0.0.1', 36812)
Executing command=PING
Command executed: PING
Key evolved: k1 → k2
```

Figure 2: Successful command execution between client and server

2. Replay Attack Detection

In this scenario, the attacker captures a valid packet and attempts to replay it. The server detects that the counter value is not fresh and rejects the message. This demonstrates the effectiveness of the counter-based replay protection mechanism.

```
Session Actions Edit View Help
Command: PING
Sent command: PING
Server: Executed: PING

Key evolved: k1 → k2
Command: GET-STATUS
Sent command: GET-STATUS
Server: Executed: GET-STATUS

Key evolved: k2 → k3
Command: 
{"counter": 3, "cmd": "GET-STATUS", "hmac": "e4a6fcb8400333704d69efdc687c5ea28cf0ada35bd45cb184147f2948331ad"}

Packet captured:
{"counter": 3, "cmd": "GET-STATUS", "hmac": "e4a6fcb8400333704d69efdc687c5ea28cf0ada35bd45cb184147f2948331ad"}
Y
Replaying captured packet...

Packet captured:
{"counter": 3, "cmd": "GET-STATUS", "hmac": "e4a6fcb8400333704d69efdc687c5ea28cf0ada35bd45cb184147f2948331ad"}
Secure channel established
Message received from ('127.0.0.1', 36812)
Executing command=START
Command executed: START

Key evolved: k0 → k1
Message received from ('127.0.0.1', 36812)
Executing command=PING
Command executed: PING

Key evolved: k1 → k2
Message received from ('127.0.0.1', 36812)
Executing command=GET-STATUS
Command executed: GET-STATUS

Key evolved: k2 → k3
Connection from: ('127.0.0.1', 43096)
Reusing existing session key
Replay attack detected (old counter used)
```

Figure 3: Replay attack detected and rejected by the server

3. Key Evolution

After each successful command execution, the session key is updated using the key evolution mechanism. This ensures that previously used keys are not reused, enhancing the security of the system.

```
Session Actions Edit View Help
Enter commands (type 'exit' to quit):

Command: START
Sent command: START
Server: Executed: START

Key evolved: k0 → k1
Command: PING
Sent command: PING
Server: Executed: PING

Key evolved: k1 → k2
Command: GET-STATUS
Sent command: GET-STATUS
Server: Executed: GET-STATUS

Key evolved: k2 → k3
Command: 

(dhroushukla@kali: ~/Desktop/AC)
$ python3 secure_server.py
Server listening...

Counter Mode: True
Key Evolution: True

Connection from: ('127.0.0.1', 36812)
Secure channel established

Message received from ('127.0.0.1', 36812)
Executing command=START
Command executed: START

Key evolved: k0 → k1
Message received from ('127.0.0.1', 36812)
Executing command=PING
Command executed: PING

Key evolved: k1 → k2
Message received from ('127.0.0.1', 36812)
Executing command=GET-STATUS
Command executed: GET-STATUS

Key evolved: k2 → k3
```

Figure 4: Continuous key evolution across multiple authenticated commands

The results clearly indicate that the system is capable of detecting and preventing replay attacks while maintaining secure and synchronized communication between the client and server. The use of HMAC ensures message integrity, while key evolution strengthens the system against potential key compromise.

SECURITY ANALYSIS

The proposed system incorporates multiple security mechanisms to protect control command communication against common network attacks. The combination of RSA, HMAC, counter-based validation, and key evolution provides a layered security approach.

1. Confidentiality

Confidentiality is ensured during the key exchange phase using RSA encryption. The session key is encrypted using the server's public key and can only be decrypted using the server's private key. This prevents unauthorized entities from obtaining the session key.

2. Message Integrity

Message integrity is achieved using HMAC with SHA-256. Each message is accompanied by a hash-based authentication code generated using a secret key. Any modification to the message results in a mismatch during verification, allowing the server to detect tampering.

3. Authentication

The use of HMAC also ensures that the message originates from a trusted source. Since only the client and server share the session key, only valid entities can generate correct HMAC values.

4. Replay Attack Prevention

Replay attacks are prevented using a counter-based mechanism. Each message contains a counter value that must be greater than the previously received value. If an attacker attempts to resend an old message, the server detects the outdated counter and rejects the request.

5. Key Evolution and Forward Security

The system implements adaptive key evolution, where the session key is updated after each successful message exchange. This ensures that even if a key is compromised, it cannot be used to authenticate future messages. This provides forward security and reduces the risk associated with long-term key usage.

6. Resistance to Man-in-the-Middle Attacks

Although the communication channel is insecure, the system is resistant to man-in-the-middle attacks due to encryption and authentication mechanisms. An attacker can intercept packets but cannot modify or replay them successfully without being detected.

ADVANTAGES AND LIMITATIONS

Advantages

- Provides secure key exchange using RSA encryption
- Ensures message integrity and authentication using HMAC
- Prevents replay attacks through counter-based validation
- Implements adaptive key evolution for enhanced security
- Detects and rejects tampered or unauthorized messages
- Simulates real-world attacks using an attacker module
- Modular design allows easy testing and scalability

Limitations

- RSA-based key exchange can be computationally expensive
- Does not use symmetric encryption (e.g., AES) for message confidentiality after key exchange
- Requires synchronization between client and server for key evolution
- Limited to a controlled environment and not optimized for large-scale deployment
- Does not include secure certificate-based authentication

FUTURE SCOPE

The current system provides a strong foundation for secure control command communication. However, several enhancements can be made to further improve its security, scalability, and real-world applicability.

- Replace RSA with Elliptic Curve Cryptography (ECC) for faster and more efficient key exchange
- Integrate symmetric encryption algorithms such as AES for encrypting message content after key exchange
- Implement digital certificates for stronger authentication between client and server
- Develop a graphical user interface (GUI) or web-based interface for easier interaction
- Extend the system to support multiple clients in a distributed environment
- Deploy the system in real-world applications such as IoT devices and industrial control systems
- Implement advanced attack detection mechanisms such as anomaly detection or intrusion detection systems

These improvements can enhance the system's performance and make it suitable for practical deployment in modern secure communication environments.

CONCLUSION

This project presents a secure client–server protocol for control command communication over an insecure network using RSA and HMAC. A session key is established securely using RSA, after which all commands are authenticated using HMAC and protected against replay attacks using a counter-based mechanism.

The system successfully ensures message integrity, authentication, and freshness. An attacker model was implemented to simulate real-world threats such as packet interception and replay attacks, and the system effectively detected and rejected malicious attempts.

Adaptive key evolution was incorporated to update the session key after each successful communication, enhancing security and preventing reuse of compromised keys. The implementation demonstrates that combining encryption, authentication, and key management techniques can provide a robust and practical solution for secure command transmission.

Overall, the project achieves its objectives and provides a solid foundation for secure communication systems in applications such as IoT, remote administration, and industrial control environments.

REFERENCES

1. Cryptography Library Documentation. Available at: <https://cryptography.io>
2. TryHackMe – Cybersecurity Training Platform. Available at: <https://tryhackme.com>
3. Hack The Box – Penetration Testing Labs. Available at: <https://www.hackthebox.com>
4. Krawczyk, H., Bellare, M., & Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication (RFC 2104).
5. Rivest, R., Shamir, A., & Adleman, L. (1978). A Method for Obtaining Digital Signatures and Public-Key Cryptosystems.
6. Python Documentation – <https://docs.python.org>